

Rover Ready

Project 2 Written Proposal

Course: Rutgers Virtual Reality & Lab, Project 2

Group: Group 14

Group Members: Chirag Baviskar and Dhrumil Patel

Date: April 20, 2026

1. Introduction

Rover Ready is a small browser-based VR repair experience set inside a single garage bay. The player starts in front of a damaged exploration rover that is almost ready for deployment, but a few important systems still need work. The main goal is to pick up a power module, insert it into the rover, and then use a handheld repair tool to fix damaged parts. After those tasks are finished, the rover powers on and the bay shifts into a more active, mission-ready state.

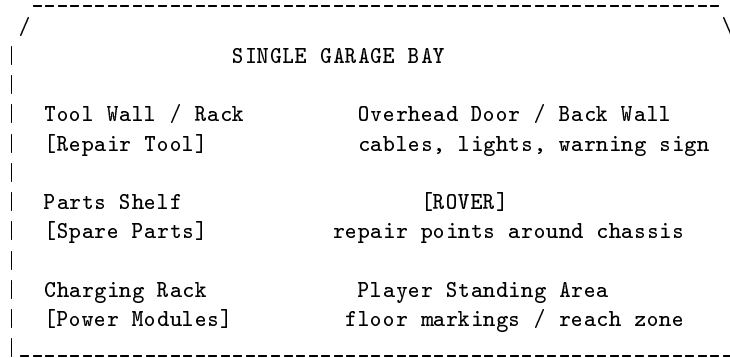
We think VR fits this idea well because the experience depends on hand movement and spatial interaction instead of menus or button prompts. Picking up a power module, lining it up with the socket, and pushing it into place should feel more natural in VR than it would on a normal screen. The repair step also benefits from depth and hand presence because the player is working directly on visible parts of the rover. Instead of trying to build a large world, we are keeping the project focused on one polished room with two interactions that should feel solid and satisfying.

2. Environment Design

The environment is a small industrial garage built around one rover service bay. The rover sits at the center of the room, and the player stays mostly in one area with the main objects placed within easy reach. Around the rover, we plan to place a charging rack for power modules, a tool wall, floor markings, a small shelf for spare parts, and a few extra industrial details so the room feels believable without becoming too large. Visually, we want an intentional low-poly style with chunky shapes, simple materials, and a limited palette built around gray metal, muted sand, warning yellow or orange, and small indicator lights.

The room should also show clear progress over the course of the interaction loop. At the beginning, the rover is inactive, the lighting is a little dimmer, and a few parts look damaged or unpowered. After the player inserts the power module and finishes the repairs, the rover lights up and the bay feels more active. That before-and-after change gives the player a sense of completion without needing multiple rooms or a larger game structure.

Rough Layout Diagram



This diagram shows one enclosed garage room with all of the main gameplay elements arranged around the rover in the same bay. The player can move between the charging rack, the repair tool, and the rover without the project turning into a larger multi-room scene, which keeps the scope practical for a two-person team.

3. Interactions and Features

The gameplay loop has two main interactions. First, the player grabs a power module from the charging rack and inserts it into a guided socket on the rover. To keep that step from feeling frustrating, we plan to use snap-target alignment so the module locks into place once it is close enough. When the module is inserted correctly, the rover powers on partially and its indicators change. Second, the player picks up a repair tool and uses it on a few damaged rover components, such as a panel, wheel mount, or antenna base. Each repair point will give a visible response so it is clear when that part has been fixed.

Feature Mapping Table

Planned Item	Type / Category	Implementation Plan
Original rover repair theme	Base requirement	Build a stylized rover maintenance scenario that is distinct from the lab and homework examples.
Custom power module asset	Base requirement	Model in Blender and export for loading with GLTFLoader .
Custom handheld repair tool asset	Base requirement	Model in Blender and export for loading with GLTFLoader .
Power module insertion	Base requirement	Use VR grabbing plus guided socket snapping and a powered-on state change on the rover.
Component repair interaction	Base requirement	Use a handheld tool to repair multiple rover targets with visual completion feedback.
Physics	Advanced feature (Category A)	Use Cannon-ES for held and dropped objects such as power modules and tools, plus interaction with the bay floor or holders.
Multiple object types / tools	Advanced feature (Category A)	Implement different behavior for modules, sockets, repair targets, and the repair tool rather than treating all objects the same way.

If we need a backup advanced feature, our best option is procedural generation for repeated environment assets such as floor panels, shelves, or wall modules. That is not part of the core plan right now, but it gives us a reasonable fallback without changing the main gameplay loop.

4. Task Division

We want the work split to stay clear from the beginning. Chirag will mainly handle the bay layout, the overall visual direction, the required custom Blender assets, and model export prep. That includes leading the power module, the repair tool, and the setup of the garage environment. Dhrumil will mainly handle the Three.js and WebXR side of the project, including scene setup, grabbing, snap-based insertion, repair logic, and Cannon-ES physics.

We may use a small number of third-party assets for background props like crates, cables, or other industrial details if that helps us save time. Those assets would only be used to fill the scene and will not count toward the required custom Blender models. Both of us will still help with playtesting, emulator testing, and final polish, but the main ownership of the art and programming work will stay clear.

5. Progress So Far

So far, we have finished the planning stage and narrowed the idea into a scope that we think is realistic to complete. We chose the **Rover Ready** concept, locked in the two main interactions, matched the project to two Category A advanced features, and divided the main responsibilities between us. We also completed the proposal presentation materials and wrote planning notes for the visual direction, interaction flow, risks, and backup options.

The project is still early in implementation, so the most important progress so far has been keeping the idea focused and finishable instead of adding extra systems too early. Our next steps are to lock the bay layout, build the first custom asset in Blender, set up the initial WebXR scene, and get a basic version of the insertion and repair loop working as soon as possible. Once that is in place, we can spend more time refining the experience instead of redesigning it.